

Guion de Presentación — Reino Olvidado: Crónicas del Abismo

Duración estimada: 8-12 minutos

Tipo de proyecto: Aplicación full-stack (Java + Web)

Tecnologías: Java 17 · Spring Boot 3 · HTML/CSS/JS Vanilla

Estructura del guion

1. Introducción (1-2 min)
 2. Qué es la aplicación (1 min)
 3. Arquitectura técnica (2-3 min)
 4. Demo en vivo (3-4 min)
 5. Código destacado (1-2 min)
 6. Cierre (30 seg)
-

1. Introducción

 **Di algo parecido a esto:**

"Buenos días/tardes. Mi proyecto se llama **Reino Olvidado: Crónicas del Abismo**, y es un videojuego RPG Roguelike de mazmorras que funciona completamente en el navegador."

"La idea surgió de querer aplicar los conceptos de **programación orientada a objetos** que hemos visto en clase — herencia, polimorfismo, interfaces — pero en un contexto real y divertido: un videojuego."

[!NOTE] Si te preguntan por qué un juego: di que los juegos son un entorno perfecto para aplicar POO porque tienen jerarquías de clases naturales (personaje → jugador, enemigo, jefe), estados, y lógica de negocio compleja.

2. Qué es la aplicación

 **Explica el concepto:**

"Reino Olvidado es un **juego por turnos** en el que el jugador elige una clase — Guerrero, Mago o Arquero — y explora mazmorras generadas de forma aleatoria."

Puntos clave a mencionar:

-  **Mapas procedurales** → cada partida es diferente
-  **Combate por turnos** → ataque, especial (con maná), usar ítems
-  **Inventario y equipamiento** → pociones, armas, armaduras
-  **Persistencia** → la partida se guarda en disco
-  **Ranking global** → las puntuaciones quedan guardadas entre sesiones

[!TIP] Muestra el menú principal mientras dices esto. El impacto visual ayuda mucho.

3. Arquitectura técnica

🗨 Esta es la parte más importante. Explícala con calma:

🔑 La aplicación tiene dos capas:

🖥 Backend — Java 17 + Spring Boot

"El backend está hecho en **Java con Spring Boot**. Spring Boot nos permite levantar un servidor web embebido sin configuración extra — basta con ejecutar el proyecto y el servidor arranca en el puerto 8080."

Módulos del backend (explica brevemente cada uno):

Paquete	Qué hace
Characters/	Jerarquía de clases: Character (abstracta) → Player , Enemy , Boss
Combat/	Motor de combate por turnos
Inventory/	Gestión de ítems: pociones, armas, armaduras
Management/	Guardado/carga con serialización binaria (game_save.dat)
Map/	Generación procedural del mapa
com/rpg/web/	API REST que conecta el backend con el frontend

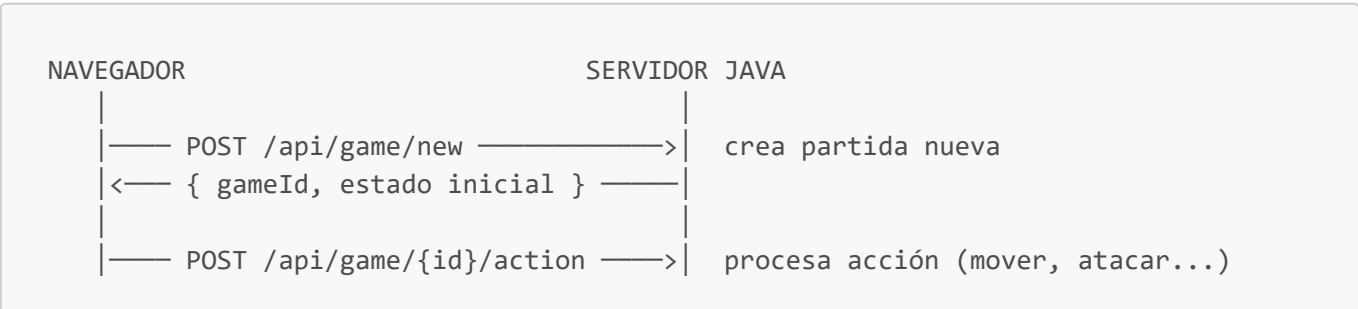
"La comunicación entre frontend y backend es mediante una **API REST** — el navegador manda peticiones HTTP y el servidor responde con JSON."

🌐 Frontend — HTML / CSS / JS Vanilla

"El frontend es una **Single Page Application** sin ningún framework externo. Todo el juego funciona en un único HTML con cinco pantallas distintas que se alternan dinámicamente."

"El JavaScript se encarga de pintar el mapa, gestionar el combate, actualizar el HUD y comunicarse con el backend."

📄 Flujo de datos (explícalo con gestos o diagrama)



```

|<— { nuevo estado JSON } —————|
|                                     |
|— GET /api/ranking —————>| consulta puntuaciones
|<— [ lista de entradas ] —————|

```

[!IMPORTANT] Menciona que el servidor mantiene el estado de la partida en memoria (sesión) y que el cliente solo renderiza lo que el servidor le manda. La lógica del juego está **100% en el servidor**.

💡 Conceptos de POO aplicados

"Los conceptos que hemos visto en clase están muy presentes:"

- **Herencia:** `Warrior`, `Mage`, `Archer` extienden `Player`, que extiende `Character`
- **Polimorfismo:** el combate llama a `especial()` sobre el personaje sin saber qué clase concreta es
- **Encapsulación:** cada clase gestiona su propio estado (HP, maná, inventario)
- **Interfaces:** `IPartyManager` define el contrato de gestión del grupo
- **Factory pattern:** `CharacterFactory` instancia el personaje correcto según la clase elegida
- **Serialización:** `FileManager` guarda y carga la partida en binario con `ObjectOutputStream`

4. Demo en vivo

🗨️ Aquí es donde demuestras el juego. Sigue este orden:

Paso a paso de la demo:

1. Arrancar el servidor (si no está corriendo)

- "Para arrancar la app ejecuto `mvnw.cmd spring-boot:run` y en unos segundos el servidor está listo."

2. Abrir el navegador en `localhost:8080`

- Comenta el diseño medieval mientras carga: "El frontend usa las fuentes `Cinzel` e `IM Fell English` para darle la estética medieval."

3. Pantalla de menú principal

- Señala las partículas animadas de fondo: "Hay pequeñas partículas animadas por CSS para dar vida al menú."
- Botones: Nueva Partida, Salón de la Gloria

4. Crear personaje

- Escribe un nombre → selecciona una clase (muestra los 3 cards brevemente)
- "Cada clase tiene stats distintos y una habilidad especial diferente."
- Pulsa **Comenzar Aventura**

5. Pantalla de juego

- Señala el mapa ASCII: "El mapa se genera proceduralmente cada partida. Los símbolos representan distintos tipos de celda: @ es el jugador, g un goblin, B un jefe, > las escaleras al siguiente piso..."
- Señala el HUD: nombre, clase, nivel, piso, turno, puntuación, barras de HP/maná/EXP
- Mueve el personaje con WASD

6. Combate

- Muévete hacia un enemigo para provocar el combate
- "Al encontrar un enemigo se abre automáticamente el modal de combate."
- Muestra: **Atacar**, **Especial** (gasta maná), **Usar Ítem**
- Gana o pierde el combate para enseñar ambas situaciones si es posible

7. Inventario (tecla I)

- "Con la tecla I puedo abrir el inventario para ver y usar ítems directamente."

8. Guardar partida (tecla X)

- "Al guardar, la puntuación se registra en el ranking y la partida se serializa en disco."
- Muestra el archivo `game_save.dat` y `adventure_log.txt` si tienes tiempo

9. Salón de la Gloria (Ranking)

- Vuelve al menú → pulsa "Salón de la Gloria"
- "El ranking persiste entre ejecuciones del servidor porque se guarda en `ranking.json`."

[!WARNING] Si algo falla durante la demo (el servidor no arranca, error en pantalla...) mantén la calma. Di: "Esto puede pasar en desarrollo. En producción se gestionaría con logs y manejo de errores más robusto." Y muestra el `server.log`.

5. Código destacado

🗨 Elige 1 o 2 fragmentos de código para enseñar (no más). Sugerencias:

Opción A — Jerarquía de personajes (POO pura)

"Aquí se ve la herencia: `Character` es abstracta y obliga a todas las subclases a implementar `performSpecial()`. El polimorfismo hace que el combate no necesite saber con qué clase concreta trabaja."

```
// Character.java – clase base abstracta
public abstract class Character {
    protected String name;
    protected int hp, maxHp, attack, defense;
    public abstract String performSpecial(Character target);
}

// Warrior.java – subclase concreta
public class Warrior extends Player {
    @Override
    public String performSpecial(Character target) {
        // Golpe Aplastante: daño doble
    }
}
```

```

        int damage = (attack * 2) - target.defense;
        target.hp -= Math.max(1, damage);
        return name + " usa Golpe Aplastante → " + damage + " daño";
    }
}

```

Opción B — API REST (endpoint de acción)

"Este es el endpoint principal. El frontend manda una acción en texto y el servidor la procesa y devuelve el nuevo estado del juego como JSON."

```

// GameController.java
@PostMapping("/{id}/action")
public ResponseEntity<GameStateDTO> action(
    @PathVariable String id,
    @RequestBody ActionRequest req) {

    WebGame game = sessions.get(id);
    game.processAction(req.getAction());
    return ResponseEntity.ok(game.getState());
}

```

Opción C — Sistema de puntuación

"La fórmula de puntuación premia el progreso: pisos explorados, nivel alcanzado y turnos sobrevividos."

```
Score = (Piso × 100) + (Nivel × 50) + (Turnos × 2)
```

6. Cierre

💬 **Termina con algo breve y seguro:**

"En resumen, Reino Olvidado es una aplicación full-stack que aplica los principios de programación orientada a objetos en un entorno real: herencia, polimorfismo, interfaces, serialización y comunicación cliente-servidor mediante una API REST."

"Ha sido un proyecto muy completo porque he tenido que diseñar tanto la lógica de negocio en Java como la interfaz en el navegador, y hacer que ambas capas se comuniquen correctamente."

"¿Alguna pregunta?"

? Posibles preguntas y cómo responderlas

Pregunta

Respuesta

Pregunta	Respuesta
¿Por qué Spring Boot y no un servidor manual?	Spring Boot abstrae la configuración del servidor (Tomcat embebido), inyección de dependencias y mapeo de rutas HTTP. Es el estándar de la industria en Java.
¿Qué es una API REST?	Una API REST es un conjunto de endpoints HTTP que siguen un estilo arquitectónico. El cliente manda peticiones con verbos (GET, POST) y el servidor responde con JSON.
¿Cómo se guarda la partida?	Con serialización binaria Java (<code>ObjectOutputStream</code>). Los objetos del juego se convierten a bytes y se escriben en <code>game_save.dat</code> . Para cargar, se leen con <code>ObjectInputStream</code> .
¿Podría jugarse en multijugador?	El servidor ya gestiona sesiones por ID de partida (<code>gameId</code>). Habría que añadir autenticación y que cada sesión esté vinculada a un usuario.
¿Por qué el frontend sin frameworks?	Para entender bien el DOM, el ciclo de vida de los eventos y la comunicación con APIs sin capas de abstracción. Es más didáctico y no añade dependencias.
¿Qué es el patrón Factory?	Es un patrón de diseño que delega la creación de objetos a una clase separada. En este proyecto, <code>CharacterFactory</code> crea el personaje correcto (<code>Warrior</code> , <code>Mage</code> , <code>Archer</code>) según el string que llega del frontend.

[!TIP] **Consejo final:** Practica la demo **al menos una vez** antes de presentar para asegurarte de que el servidor arranca sin problemas y la partida fluye. Si puedes, ten el servidor ya corriendo cuando empiece la presentación.